

Bloom Filter

Approximate membership query

Bloom Filter

- Space-efficient
- Membership query – Contains (x)
- Positive/Negative result
- False Positive results.
- But no False Negative result.
- a query returns either "possibly in set" or "definitely not in set"

Probabilistic hashing

- Bloom filter – use many hash function
- Reduce space-complexity for “Big Data”
- Variations: Counting bloom filter, Spatial bloom filter, etc.
- Source: Book chapter, Wikipedia

Algorithm for adding an element to the set

- An *empty Bloom filter* is a [bit array](#) of m bits, all set to 0.
- There must also be k different [hash functions](#) defined, each of which [maps](#) or hashes some set element to one of the m array positions, generating a uniform random distribution.
- To *add* an element, feed it to each of the k hash functions to get k array positions.
Set the bits at all these positions to 1.

Illustration of “insert a key” to bloom filter

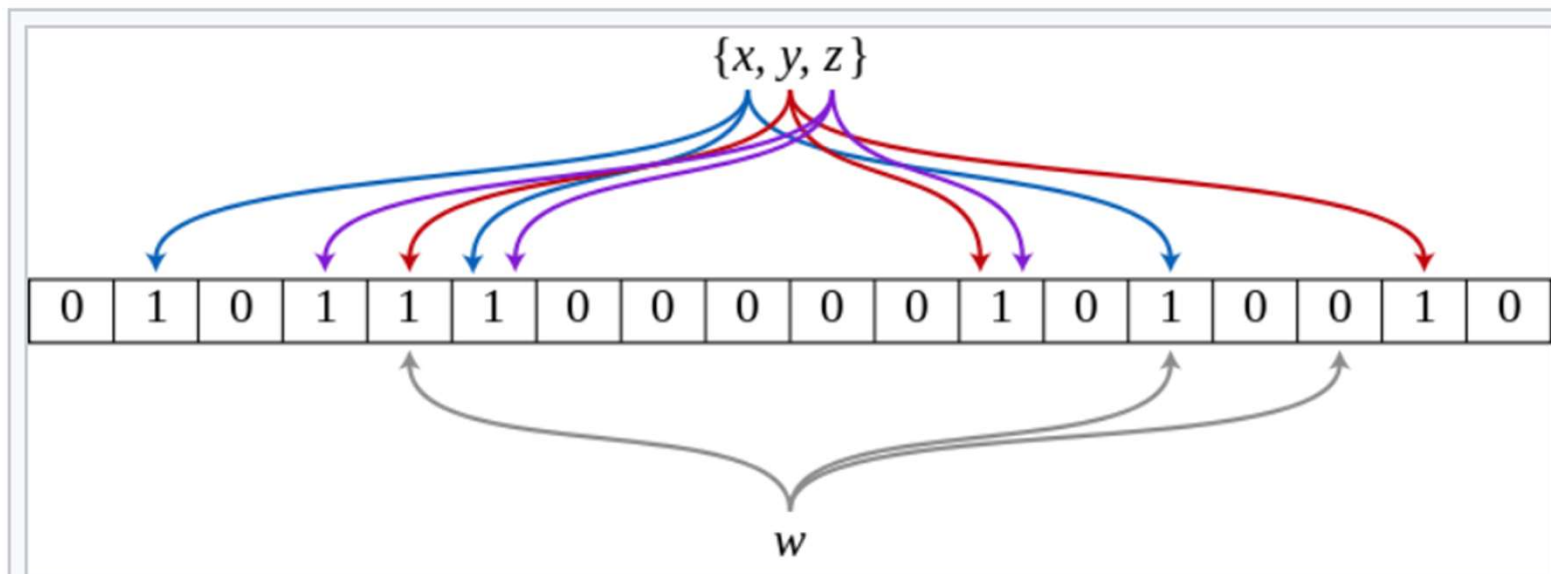
[illegible]

Bloom Filters by Example

- Interactive example here:

<https://lmlib.github.io/bloomfilter-tutorial/>

Bloom filter demo



An example of a Bloom filter, representing the set $\{x, y, z\}$.

The colored arrows show the positions in the bit array that each set element is mapped to. The element w is not in the set $\{x, y, z\}$, because it hashes to one bit-array position containing 0. For this figure, $m = 18$ and $k = 3$.

Membership Query

- To *query* for an element (test whether it is in the set), feed it to each of the k hash functions to get k array positions.
- If any of the bits at these positions is 0, the element is definitely not in the set.
- If all are 1, then the element may be in the set.
- false positive is possible – the bits have by chance been set to 1 during the insertion of other elements

Bloom Filter properties

- Elements can be added to the set, but not removed.
- The more elements that are added to the set, the larger the probability of false positives.
- Storage requirement : e.g. fewer than 10 bits per element are required for a 1% false positive probability.
- False positives depend on number of bits, # hash functions and how many elements are inserted.
- Error Probability is based on fraction of 1's in the bit array.

Interesting Properties

- *Time needed either to add items or to query is a fixed constant, $O(k)$, completely independent of the number of items already in the set.
 - In a hardware implementation, however, the Bloom filter shines because its k lookups are independent and can be parallelized.
- * Unusual property unlike many other data structures.

Applications of Bloom Filter

- **Hbase, Cassandra, Postgres**

Select * from Table where id = x.

Use Bloom filter first for checking the **id** and then the database.

This works well for non-existent rows.

- **Malicious website:** Google keeps track of the malicious websites URL (50 MB). Instead of sending the actuals URLs to Google Chrome Browser, it sends the Bloom Filter (1 MB) and hash functions to all the browsers.
- **Medium** website maintains a bloom filter for each user so that it does not redundantly recommend a website to user.

How big should a bloom filter be?

- A larger filter will have less false positives, and a smaller one more.
- How to control false positive rate
- Configurable based on application.

Deriving False Positive Rate

- Probability
- Based on Bloom Filter Chapter
- Tradeoff between space and accuracy

Probability basics

- What is the probability of getting a 4 in a throw of dice?
- What is the probability of getting an odd number in a throw of dice?

Probability basics

- What is the probability of getting a 4 in a throw of dice?

$$1/6$$

- What is the probability of getting an odd number in a throw of dice?

$$3/6 = 1/2$$

Probability basics

- What is the probability of getting a 4 and then an odd number in two successive throws of dice?

$$1/6 * 1/2 = 1/12$$

Probability basics

- What is the probability of getting a 4 and then an odd number in two successive throws of dice?

$$1/6 * 1/2 = 1/12$$

- The probability that two independent events both occur equals the product of their individual probabilities.

Probability basics

- If probability of event A is $1/6$
- Total probability is 1
- Probability of event A not happening is $1 - 1/6 = 5/6$

Assumptions

- Uniform distribution of keys in the bit array.
- Analysis will not be exact because of assumption.
- Approximate and practically useful

Probability calculation

- Suppose a dataset **S** is inserted into a bloom filter that uses **m** hash functions and length-**n** bit array.

What is the probability that the array's first bit is set to 1 ?

Probability calculation

- Suppose a dataset S is inserted into a bloom filter that uses m hash functions and length- n bit array.

What is the probability that the array's first bit is set to 1 ?

Throwing a dart



Array A



n elements space available

Dart Analogy: Setting a bit to 1 on insertion

Assume there are n places a dart can land.

What is the probability that a dart hits the first region $A[0]$?

$1/n$

What is the probability that the dart will miss the first region?

$1 - 1/n$

Dart Analogy: Setting a bit to 1 on insertion

Assume there are n places a dart can land.

What is the probability that a dart hits the first region $A[0]$?

$1/n$

What is the probability that the dart will miss the first region?

$1 - 1/n$

What is the probability that **every** dart will miss the first region?

$(1 - 1/n)^{m|S|}$

product of individual probabilities

m is number of hash functions

$|S|$ is size of set inserted to bloom filter

Dart Analogy: Setting a bit to 1 on insertion

Probability that some dart hits the first region

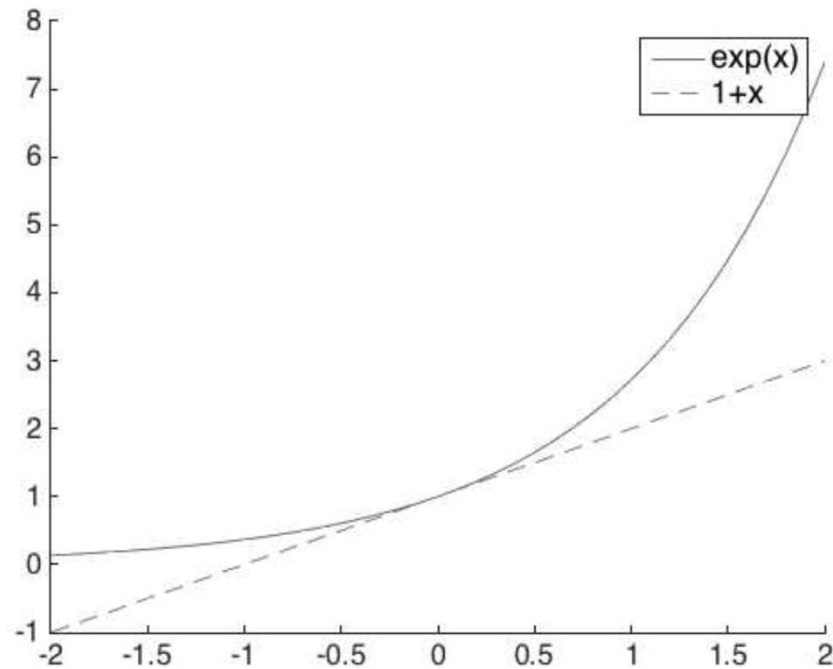
= probability that the 1st bit of bloom filter is set to 1

$$= 1 - (1 - 1/n)^{m|S|}$$

Simplify this expression for ease of calculation

$$1 - (1 - 1/n)^{m|S|}$$

e^x is a good approximation of $1 + x$ when x is close to 0. $e = 2.71$



Estimate of probability that a given bit is 1

e^x is a good approximation of $1 + x$ when x is close to 0. $e = 2.71$

$$\begin{aligned} & 1 - (1 - 1/n)^{m|S|} \\ = & 1 - (e^{-1/n})^{m|S|} && \text{per key storage } \mathbf{b} = n/|S| = \text{number of bits in array} / \text{size of set} \\ = & 1 - (e^{-|S|/n})^m \\ = & 1 - (e^{-1/b})^m \\ = & 1 - (e^{-m/b}) \end{aligned}$$

Estimate of False positive probability

- A false positive occurs for a key not in S if all the m bits are 1 after inserting the keys in S
- Probability that a given bit is 1 is approximately $(1 - e^{-m/b})$
- Probability that all m of these bits are set to 1 is approximately $(1 - e^{-m/b})^m$
- Sanity check
 $m = b = 1$ and $b \rightarrow \text{infinity}$

Tradeoff between space and accuracy

- Accuracy depends on
 - m = number of hash functions and
 - b = per key space

$$(1 - e^{-m/b})^m$$

What is the best choice for m and b?

Tradeoff between space and accuracy

- Accuracy depends on
m = number of hash functions and
b = per key space
 $(1 - e^{-m/b})^m$

What is the best choice for m and b?

Using calculus, optimal choice of m = $(\ln 2) \cdot b$

How many hash functions?

E.g., if b = 8, m = $(\ln 2) \cdot 8$ = approx. 5

$$b = n / |S| = 1000/125$$

False positive rate in terms of space requirement

m = number of hash functions and

b = per key space

$$(1 - e^{-m/b})^m$$

For optimal choice of m , false positive rate

$$\begin{aligned} & (1 - e^{-\ln 2})^{\ln 2 \cdot b} \\ &= (1 - 1/e^{\ln 2})^{\ln 2 \cdot b} \\ &= (1 - 1/2)^{\ln 2 \cdot b} \\ &= (1/2)^{\ln 2 \cdot b} \end{aligned}$$

False positive rate in terms of space requirement

m = number of hash functions and

b = per key space

$$(1 - e^{-m/b})^m$$

For optimal choice of m , false positive rate

$$= \left(\frac{1}{2}\right)^{\ln 2 \cdot b} \quad \text{More space (b), less false positives}$$

Estimate when $b = 8$ bits

$$\begin{aligned} &= \left(\frac{1}{2}\right)^{\ln 2 \cdot 8} = (0.5)^{5.54} \\ &= 0.023 = \text{about 2 \% error rate} \end{aligned}$$

Problem 12.3 We interpreted our heuristic analysis of bloom filters in Section 12.6 by specializing it to the case of 8 bits of space per key inserted into the filter. Suppose we were willing to use twice as much space (16 bits per insertion). What can you say about the corresponding false positive rate, according to our heuristic analysis, assuming that the number m of hash tables is set optimally? (Choose the strongest true statement.)

- a) The false positive rate would be less than 1%.
- b) The false positive rate would be less than 0.1%.
- c) The false positive rate would be less than 0.01%.
- d) The false positive rate would be less than 0.001%.

Answer

Solve the expression with $b = 16$

$$\left(\frac{1}{2}\right)^{\ln 2 \cdot b}$$

Probability of false positive out of 1 is 0.000488

In percentage (in 100 %) = .04 %

This is less than 0.1%